ExpenseMate - Metrics Report

Version 1.0. Prepared by: Syed Arham (24K-2551) - Member A
and Muhammad Huzaifa (24K-3034) - Member B.

This report records the size, test, code complexity and fault metrics of ExpenseMate v1.0.0. Size and test-plan figures come from the project description; complexity and fault figures must be filled in from your own measurements.

1. Purpose and Method

The metrics support Phase 4 of the assignment: measuring code complexity (Member A) and tracking fault-find and fault-fix rates (Member B). Cyclomatic complexity (CC) of a function is its number of decision points plus one, or E - N + 2P on its control-flow graph.

2. Size Metrics

Area	Measure	Value
Frontend	Screens	22
Frontend	Reusable components	6
Frontend	Models	5
Frontend	Services (api, auth, expense state, currency)	4
Frontend	Utility files	4
Backend	Controllers	7
Backend	Route modules	7
Backend	Services (codes, export, mailer)	3
Backend	Utility modules	5
Backend	Middleware modules	2
Backend	REST endpoints	29
Database	Tables	6

Area	Measure	Value
Database	Default categories and accounts seeded per user	13 and 5

To fill the two line-count rows, run cloc lib and cloc backend/src, and record the code lines.

3. Test Metrics

Test level	Planned cases	Executed	Passed	Pass rate
Backend unit	67	20	20	100%
Frontend unit	46	10	10	100%
Integration	40	8	8	100%
System	35	12	12	100%
Total	188	50	50	100%

Pass rate = passed / executed x 100. Execution counts come from the Status column of the Test Cases Document.

4. Cyclomatic Complexity

4.1 How to measure

- Backend (JavaScript): run lizard backend/src, or enable the ESLint complexity rule with a threshold of 1 so every function is reported.
- Frontend (Dart): use Dart Code Metrics, or count decision points (if, else if, case, for, while, &&, ||, ?:, catch) and add one.
- Record the highest CC and the average CC for each file.

4.2 Risk scale

CC	Risk	Action
1 to 10	Low	No action
11 to 20	Moderate	Review; add tests

CC	Risk	Action
21 to 50	High	Refactor into smaller functions
Above 50	Very high	Untestable; must be refactored

4.3 Results

Module	File	Highest CC (function)	Average CC	Risk
Auth controller	controllers/auth.controller.js			
Transaction controller	controllers/transaction.controller.js			
Budget controller	controllers/budget.controller.js			
Analytics controller	controllers/analytics.controller.js			
User controller	controllers/user.controller.js			
Account and category controllers	controllers/account.controller.js, category.controller.js			
OTP service	services/codes.js			
Export service	services/export.service.js			
Validator	utils/validator.js			
Date utilities	utils/dates.js			

Module	File	Highest CC (function)	Average CC	Risk
Auth and error middleware	middleware/auth.js, errorHandler.js			
Expense state	services/expense_state.dart			
API service	services/api_service.dart			
Add transaction page	pages/transaction/add_transaction_page.dart			
Budget page	pages/budget/budget_page.dart			
Analytics page	pages/analytics/analytics_page.dart			
Profile page	pages/profile/profile_page.dart			

5. Fault Find and Fault Fix Tracking

5.1 Formulas

Metric	Formula
Fault find rate	Faults found / test cases executed
Fault fix rate	Faults fixed / faults found x 100
Defect density	Faults found / thousand lines of code
Mean time to fix	Total fixing hours / faults fixed

Severity levels: Critical (system unusable), High (feature fails), Medium (feature works with a workaround), Low (cosmetic). Status values: Open, Fixed, Verified, Closed.

6. Conclusion

After the tables are filled, summarise in three sentences: which modules had the highest complexity, how many faults were found and fixed, and what was refactored as a result.